중간발표 대본 (10분 판) — 지도 기반 노선 타당성 분석 시뮬레이터

컴퓨터시스템공학과 3학년 2학기 · 202245079 신선호 · 심화응용프로젝트

장마다 숫자 하나와 배운 것 하나만 남겼습니다. 자세한 설명은 전문 대본(약 27분)에 있습니다.

장	내용	초
1	표지	33
2	문제	28
3	사용 흐름	28
4	실제 화면	49
5	시스템 구조	23
6	기술 스택	18
7	외부 자료	44
8	데이터 파이프라인	25
9	비용 모델	37
10	수요 모델	39
11	B/C 산출	52
12	추천 알고리즘	41
13	정차역 자동 배치	39
14	검증	31
15	현재 상태	39
	합계	**8분 48초**

말하는 시간만 8분 48초입니다(1분에 300자, 공백 제외). 장을 넘기고 숨 고르는 시간을 장마다 3~4초 더하면 약 10분입니다 — 1분쯤 여유를 남겼습니다. 소리 내어 한 번 읽어 보고, 빠르면 문장 끝에서 한 번씩 더 쉬면 됩니다.

1. 표지 — RailFeas

약 33초

안녕하십니까, 컴퓨터시스템공학과 신선호입니다. RailFeas 는 지도에 노선을 그으면 그 노선을 지을 만한지 숫자로 답하는 웹 서비스입니다. 건설비는 실제 철도사업 38건, 수요는 수도권 514개 역에서 회귀로 얻었고, B/C 원단위는 국토교통부 투자평가지침을 따랐습니다. 오늘은 사용 흐름과 구조, 비용·수요 모델, B/C, 추천 알고리즘, 검증 순서로 말씀드리겠습니다.

2. 문제 — 지을 만한가에 답하려면 몇 달

약 28초 · 화면에서 짚을 곳: 오른쪽 어두운 칸

예비타당성조사는 몇 달이 걸리지만, 후보를 추리는 초기 단계에 필요한 건 정확한 한 값이 아니라 후보 사이의 순위입니다. 그래서 입력은 선 하나, 출력은 수단 열 개의 B/C 비교표입니다. 모델 오차는 비용 1.25배, 수요 1.75배이고 화면에 그대로 적었습니다. 절대값을 믿으라는 서비스가 아니라, 순위를 보라는 서비스입니다.

3. 사용 흐름 — 클릭 두 번에서 비교표까지

약 28초 · 화면에서 짚을 곳: 아래 회색 칸

노선을 그리고 저장해 비용 비교하기를 누르면 수단 열 개를 한 번에 계산하고 정차역을 자동으로 놓습니다. 중요한 건 아래 칸입니다. 이미 있는 도시철도와 KTX 를 같은 표에 한 줄로 넣어, 추가 건설비가 0 인 대안과 견주게 했습니다. 신설안끼리만 비교하면, 이미 길이 있는 곳에 또 짓는 안이 1등으로 올라와도 보이지 않기 때문입니다.

4. 실제 화면 — 시흥~안양 14km

약 49초 · 화면에서 짚을 곳: 왼쪽 표의 맨 위와 지하철 지하 행

시흥~안양 14킬로미터를 실제로 계산한 화면입니다. 정차역이 자동으로 놓이는데, 역 이름은 제가 적은 게 아니라 역을 놓은 자리의 행정동 이름입니다. 같은 노선인데 BRT 고급형은 629억에 B/C 2.05, 지하철 지하는 1조 9천억에 0.15 로, 수단에 따라 건설비가 30배 갈립니다. 맨 아래 B/C 0 인 BRT 저급형은 계산 실패가 아니라 시내버스보다 느려 아끼는 시간이 없다는 결과입니다. 오른쪽은 노선끼리 견주는 화면입니다. 추천안 B/C 는 시흥~안양 2.05, 서울~부산 0.07, 광명~화성 0.01 이고, 서울~부산에는 기존 KTX 가 함께 뜹니다.

5. 시스템 구조 — 세 덩어리

약 23초 · 화면에서 짚을 곳: 아래 파란 띠

구조는 React 지도 화면, Spring Boot 계산 서버, MySQL 세 덩어리입니다. 회귀는 Python 으로 미리 돌려 기준값으로 넣어 두고, 관리자가 화면에서 고칩니다. 요청마다 회귀를 돌리면 느리고, 관리자가 기준값을 고칠 수 있어야 하기 때문입니다.

6. 기술 스택 — 무엇을 왜 썼는가

약 18초 · 화면에서 짚을 곳: 표만 보여 주고 넘어간다

도구는 표와 같습니다. 스키마는 SQL 이 만들고 JPA 는 검증만 하게 해서, 둘이 어긋나면 아예 기동이 안 됩니다. VWorld 지도 타일은 Vite 프록시로 받아 인증키가 화면에 드러나지 않게 했습니다.

7. 외부 자료 — 공공 API 일곱 곳

약 44초 · 화면에서 짚을 곳: 아래 붉은 칸

자료는 공공 API 일곱 곳에서 받았습니다. OSM 에서 지방 역 좌표를 받아 지방 노선도 인구를 셀 수 있게 넓혔습니다. 아래 칸이 가장 많이 배운 부분입니다. 막혀 있던 KTX 소요시간을 국토교통부 TAGO 열차정보로 채웠습니다. 서울~부산은 44편 중앙 164분입니다. 그런데 등급 코드를 하나만 걸어 강릉선이 통째로 빠졌고, 역 이름이 두 자료에서 달랐고, 2분 차이를 계통이라 부르기도 했습니다. 세 번 모두 오류 없이 조용했습니다. 그래서 몇 개를 받았는지가 아니라 무엇이 안 왔는지를 세도록 바꿨습니다.

8. 데이터 파이프라인 — 원자료가 기준값이 되기까지

약 25초 · 화면에서 짚을 곳: 네 칸을 손으로 훑고 넘어간다

원자료는 수집, 정제, 회귀, 적재 네 단계를 거칩니다. 이상치는 지우지 않고 표시만 하고, 추정치는 학습에서 뺍니다. 지우면 왜 뺐는지 나중에 설명할 수 없고, 추정으로 만든 값으로 다시 추정하면 순환 오류가 생기기 때문입니다. 표본은 381건을 모았지만 사업비가 공시된 38건만 학습에 씁니다.

9. 비용 모델 — 실제 철도사업 38건

약 37초 · 화면에서 짚을 곳: 식 아래 줄과 회색 칸

비용은 사업비가 공시된 38건으로 로그 회귀를 돌렸고 오차는 1.25배입니다. 연장이 두 배면 사업비는 1.57배라 규모의 경제가 보이고, 전 구간을 지하로 지으면 1.78배로 계수 중 가장 큽니다. 회귀식은 관리자가 단가를 고칠 수 있게 고정비 더하기 연장 곱하기 단가 꼴로 바꿔 넣었습니다. 급행을 따로 가를지도 검정했는데, 표본이 4건이라 1.5배 차이도 못 잡는 검정이었습니다. 그래서 차이가 없다가 아니라 못 본다고 적었습니다.

10. 수요 모델 — 수도권 514개 역

약 39초 · 화면에서 짚을 곳: 왼쪽 칸과 아래 칸

수요는 514개 역으로 회귀했습니다. 인구만으로는 결정계수가 0.32 였고, 종사자와 환승 노선을 넣어 0.551 로 올렸습니다. 환승 노선이 두 배면 이용객은 2.11배로, 네 변수 중 기여가 가장 큽니다. 서울 시청과 부산 시청이 이름이 같아 한 역으로 묶이는 버그도 있었고, 도심 거리는 서울시청이 아니라 가장 가까운 광역시 도심으로 잽니다. 오차 1.75배는 크지만, 같은 수요를 모든 수단에 똑같이 쓰므로 어느 수단이 낫느냐는 결론은 바뀌지 않습니다.

11. B/C 산출 — 원단위는 지침에서

약 52초 · 화면에서 짚을 곳: 오른쪽 아래 칸

B/C 원단위는 전부 투자평가지침 제7판에서 가져왔습니다. 시간가치는 광역도시철도 등급을 골랐습니다. 업무 통행이 4퍼센트뿐이라 고속철도 값을 쓰면 편익이 부풀려집니다. 오른쪽 아래가 최근에 고친 부분입니다. 처음엔 이용객 모두가 노선 끝까지 탄다고 봐서, 서울~부산 광역급행이 기존 KTX 보다 한 시간 넘게 느린데도 B/C 가 3.86 이었습니다. 서울시 자료의 지하철 1회 평균 이동 13.6킬로미터만큼만 탄다고 고쳐 지금은 0.15 입니다. 13.6킬로미터만 타는 사람은 역이 드문 KTX 를 탈 수 없어 KTX 는 기준에서 뺐고, 기존 도시철도가 있으면 그보다 빨라진 만큼만 편익으로 셉니다.

12. 추천 알고리즘 — 20개 조합에서 하나

약 41초 · 화면에서 짚을 곳: 4단계와 오른쪽 칸

추천은 스무 조합을 다 계산한 뒤, 수송능력을 넘거나 B/C 가 0 이거나 권장 연장을 넘는 안을 걸러 B/C 최대를 고릅니다. 서울~부산에 340킬로미터짜리 BRT 가 1위로 올라와서 생긴 규칙입니다. 표에서 지우지는 않고, 기본으로 내세우지만 않습니다. 선택을 막지 않는 것이 이 서비스의 방식입니다. B/C 0 인 안은 아무 시간도 아끼지 못하는 안이라, 전부 0 이면 추천할 신설안이 없다고 적습니다. 지도, 표의 굵은 행, 서버 추천, 시나리오 비교 네 곳이 같은 규칙을

씁니다.

13. 정차역 자동 배치 — 속도는 정차 횟수가 정한다

약 39초 · 화면에서 짚을 곳: 맨 위 식과 아래 붉은 칸

운영 노선 18개를 보니 표정속도의 93퍼센트가 역간격으로 설명됐습니다. 속도를 정하는 건 기술이 아니라 몇 번 서느냐라서, 정차역 배치가 B/C 에 직결됩니다. 역간격은 수단마다 운영 노선의 중앙값을 씁니다. 지하철 1.06킬로미터, 광역급행 6.92킬로미터입니다. 서울~부산에 역이 9개만 놓인 버그도 있었는데, 자료를 안 모은 지역을 사람이 없다고 읽은 탓이었습니다. 인구가 0인 것과 인구를 모르는 것은 다릅니다. 고쳐서 32개가 됐습니다.

14. 검증 — 테스트는 통과시키려고 쓰는 게 아니다

약 31초 · 화면에서 짚을 곳: 왼쪽 아래 칸

테스트는 백엔드 99개, ETL 224개입니다. 개수보다 중요한 건, 규칙을 일부러 지워 보고 그 테스트만 깨지는지 확인하는 것입니다. 안 깨지면 아무것도 지키지 않는 테스트입니다. KTX 계통을 가르는 규칙을 지웠는데도 테스트가 통과한 적이 있습니다. 예로 든 자료가 다른 규칙에 먼저 걸렸던 겁니다. 설명서의 숫자가 코드와 다르면 깨지는 테스트도 두었습니다.

15. 현재 상태 — 기능 8개 구현

약 39초 · 화면에서 짚을 곳: 남은 것 목록과 아래 칸

제안서 기능 여덟 개를 모두 구현했습니다. 남은 것은 숨기지 않았습니다. 강을 돌아가는 선형과 교량 할증, BRT 실측 속도입니다. 이 프로젝트에서 가장 많이 배운 것은, 값을 못 구했을 때 그럴듯한 숫자로 채우는 게 가장 위험하다는 것입니다. 못 채운 칸은 비우고 이유를 적었습니다. KTX 소요시간을 처음엔 자료가 없다고 적었는데 사실은 제가 안 받은 것이었습니다. 자료가 없다와 내가 안 받았다는 다른 말입니다. 이상으로 발표를 마치겠습니다. 감사합니다.